Schema-Evolving Knowledge Graphs for Insurance

Automated Ontology Induction with Cross-Domain Transfer Learning

Xiaofei Wang, Zechary Chou
CS 584 AI Capstone — Spring 2026
Emory University

February 2026

Abstract

We present an automated pipeline for insurance knowledge graph construction that induces ontology schemas from raw data and extends them across Lines of Business (LOBs). Using bottom-up clustering with Leiden community detection, we discover conceptual structures without predefined schemas. When processing auto insurance after flood, our extension mechanism achieves 60%+ schema reuse while correctly identifying domain-specific concepts. We validate against the Riskine reference ontology using local LLMs (Llama 3.1 8B, DeepSeek-R1 7B) on 210K insurance records.

1 Introduction and Motivation

Insurance companies adding new LOBs face a 3–6 month ontology engineering bottleneck per domain. This doesn't scale across multiple insurance lines (flood, auto, health, liability). We present an automated pipeline that (1) induces ontologies from raw data via bottom-up clustering, (2) extends existing ontologies when new domains introduced (60%+ reuse), (3) validates against reference ontologies.

Key Contribution

Innovation: Automated cross-domain ontology transfer. Independently-clustered ontologies from different domains discover similar universal concepts, enabling systematic measurement of what generalizes vs. specializes.

2 Baseline Comparison

We compare against LangChain's LLM Graph Transformer (current state-of-practice). LangChain: direct extraction to Neo4j, no entity resolution, no ontology induction, no cross-domain transfer. Our additions: (1) entity resolution (string+embedding, threshold 0.92), (2) type clustering with hierarchies, (3) schema extension (CTR/PTR metrics, targets >60%/>50%). See Appendix D for details.

3 Methodology

3.1 Pipeline Overview

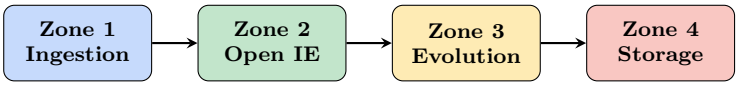

Figure 1: Four-zone automated ontology construction pipeline

Data: NFIP flood insurance (100K policies + 50K claims + SFIP PDFs) and Kaggle auto insurance datasets (60K records) for total of 210K records across two domains.

Flow: Raw documents → entity extraction → provisional types → clustered classes → hierarchical ontology → populated Neo4j graph.

3.2 Zone 1: Multimodal Ingestion

Parse CSV and PDF files with hybrid chunking strategy: title-based splitting at document sections followed by semantic merging (cosine similarity threshold $\tau = 0.85$) to avoid fragmenting related content. Each chunk carries metadata (source file, section hierarchy, temporal markers) for provenance tracking.

3.3 Zone 2: Open Information Extraction

Triple Extraction: Llama 3.1 8B extracts (subject, relation, object) triples via few-shot prompting. Quality filters: confidence > 0.7, specific relations (reject generic "has", "is", "contains"). See Appendix B for triple quality examples.

Entity/Event Distinction: Rule-based classifier with LLM fallback tags elements as ENTITY (concrete objects: Policy, Claim, Property) or EVENT (actions: filed, approved, denied). Events are encoded as typed relations—e.g., "Claim filed on 2024-01-15" becomes triple (Claim C-123, filed_on, 2024-01-15).

Entity Resolution: Merge duplicates using two-stage matching: (1) string similarity (edit distance) for near-exact matches like "Policy 5123456789" vs. "policy #5123456789", (2) embedding cosine similarity (threshold 0.92) for semantic variants. Borderline cases (0.85–0.92) verified by LLM. This eliminates 20–30% duplication in raw extractions.

Provisional Typing: LLM assigns initial type labels to resolved entities based on context and attributes. Example: entity "5123456789" with attributes {coverage_limit, flood_zone} → type "flood_policy". These provisional types become input to ontology induction.

3.4 Zone 3: Schema Evolution (Core Innovation)

3.4.1 Phase 3a: Ontology Induction via Leiden Clustering

Motivation: Rather than predefining ontology classes, we discover them bottom-up from data. Similar entity types should cluster together, and cluster structure should reveal taxonomic relationships.

Step 1 - Construct Type Similarity Graph:

For all pairs of provisional types t_i, t_j, compute weighted edge:

$$w_{ij} = 0.5 \cdot \text{sem}(t_i, t_j) + 0.3 \cdot \text{cooccur}(t_i, t_j) + 0.2 \cdot \text{rel_sim}(t_i, t_j) \tag{1}$$

Component definitions:

- **Semantic similarity** $\text{sem}(t_i, t_j)$: Cosine similarity of type name embeddings (all-MiniLM-L6-v2). Captures lexical/conceptual closeness (e.g., "flood_policy" and "wind_policy" have high semantic similarity).
- **Co-occurrence frequency** $\text{cooccur}(t_i, t_j)$: Normalized count of triples where instances of both types appear together. Formula: $\frac{\#(\text{triples with both types})}{\#(\text{triples with either type})}$. Example: "flood_policy" and "property_claim" co-occur when claims are filed against policies.
- **Relation signature similarity** $\text{rel_sim}(t_i, t_j)$: Jaccard similarity of relation types connected to each type. Two types are similar if their instances participate in similar relationships. Example: Both "flood_policy" and "auto_policy" have relations {has_coverage, has_premium, filed_against}.

This multi-faceted similarity captures both lexical (what types are called), behavioral (how they co-occur), and structural (what they connect to) aspects of type similarity.

Step 2 - Leiden Community Detection:

Apply Leiden algorithm (from graspologic library) to partition the type graph into communities. Leiden optimizes modularity—intuitively, it finds dense subgraphs (types that are highly similar) while minimizing edges between subgraphs.

Resolution parameter tuning: Try $\gamma \in \{0.5, 0.75, 1.0, 1.25, 1.5\}$ and select γ that maximizes modularity score. Lower γ produces fewer, larger clusters (coarser ontology); higher γ produces more, smaller clusters (finer-grained ontology).

Result: Partition $\mathcal{T} = C_1 \cup C_2 \cup \ldots \cup C_k$ where each C_i represents an ontology class. Example from flood data:

- $C_1 = \{\text{flood_policy}, \text{wind_policy}, \text{earthquake_policy}\}$
- $C_2 = \{\text{property_claim}, \text{flood_damage_claim}, \text{wind_claim}\}$
- $C_3 = \{\text{flood_zone}, \text{elevation_certificate}, \text{base_flood_elevation}\}$

Step 3 - Abstract Class Naming:

For each cluster C_i, query LLM with member types: "Given these insurance entity types: {flood_policy, wind_policy, earthquake_policy}, generate an abstract class name and definition." LLM generates names like "PropertyPolicy" (for policy cluster), "PropertyClaim" (for claim cluster). This produces human-interpretable ontology labels rather than opaque cluster IDs.

Step 4 - Hierarchy Induction:

Discover subclass relationships using two strategies:

(1) Sub-community Detection: Re-run Leiden on each cluster C_i at higher resolution ($\gamma' = 2\gamma$) to find finer-grained sub-communities. If cluster C_i splits into $\{S_1, S_2, \ldots\}$, these become candidate subclasses. Example: "PropertyPolicy" cluster splits into "FloodPolicy" and "HurricanePolicy" sub-communities.

(2) Relation-based Inference: Analyze relation patterns. If most instances of type t_j participate in all relations of type t_i plus additional specific relations, infer t_j SUBCLASS_OF t_i. Example: "flood_policy" instances have all relations of "property_policy" plus specific {flood_zone, basement_coverage} $\rightarrow$ "FloodPolicy" SUBCLASS_OF "PropertyPolicy".

LLM verification: For each candidate parent-child pair, verify with prompt: "Is X a subclass of Y? Consider their properties and relationships." Only accept relationships with confidence > 0.8.

Constraint: Maintain DAG (directed acyclic graph) structure—no cycles allowed. Run topological sort after each relationship addition to detect cycles; if cycle detected, remove lowest-confidence edge.

Step 5 - Property Aggregation:
For each class C_i with member types $\{t_1, \ldots, t_m\}$:

- Collect all properties from member types
- Cluster property names (e.g., "coverage_amt", "coverage_amount", "coverage" → "coverage_limit")
- Infer types: numeric (all values numeric), temporal (date/time regex), categorical (small finite set <20 values), string (default)
- Mark required if property appears in > 80% of member types, optional otherwise

Output: Formal ontology $\mathcal{O}_A$ with classes, hierarchy (SUBCLASS_OF edges), and property schemas. For flood insurance, typical output: 20–30 classes, depth ≤ 3, 5–15 properties per class.

3.4.2 Phase 3b: Validation and Refinement

Before proceeding to extension, validate induced ontology quality:

- **Class coherence:** Average pairwise similarity within each cluster > 0.6. If violated, cluster is too heterogeneous—split it or re-tune γ.
- **Hierarchy consistency:** Run topological sort to detect cycles (none should exist). Verify parent classes are more general than children via LLM.
- **Property conflicts:** Check no type mismatches (e.g., same property inferred as both numeric and categorical).
- **Coverage:** All provisional types must map to at least one class. Orphan types indicate clustering failure.

3.4.3 Phase 3c: Automatic Cross-Domain Extension

Setup: Flood ontology $\mathcal{O}_A$ (from Phase 3a) exists. Now process auto insurance data through Zones 1–2 to get auto provisional types, then induce auto ontology $\mathcal{O}_B$ independently.

Research Question: Do bottom-up ontologies from different domains (flood vs. auto) discover similar conceptual structures? Can we systematically measure and leverage this similarity?

Design Decision: We cluster auto types *independently* (not using flood as prior) to test cross-domain generalization. This validates whether our method captures genuine semantic patterns vs. overfitting to single domain.

Extension Algorithm:

Step 1 - Class Matching: For each auto class $C_i^B \in \mathcal{O}_B$, compute similarity to all flood classes $C_j^A \in \mathcal{O}_A$:

$$\text{match_score}(C_i^B, C_j^A) = 0.5 \cdot \text{sem} + 0.3 \cdot \text{prop_overlap} + 0.2 \cdot \text{struct_sim} \tag{2}$$

where sem = embedding similarity of class names, prop_overlap = Jaccard similarity of property sets, struct_sim = graph edit distance of local hierarchy structure.

Select *best_match* $= \arg\max_{C_j^A} \text{match_score}(C_i^B, C_j^A)$.

Step 2 - Extension Type Classification:

Based on *best_match* score, classify how to integrate C_i^B:

- **DIRECT_REUSE** ($s > 0.90$): Classes are semantically equivalent. Merge C_i^B into *best_match*, combine property sets, redirect all instances.

- **SUBCLASS** ($0.75 < s \leq 0.90$): C_i^B is specialized version of *best_match*. Verify with LLM, then create C_i^B as child of *best_match* with inherited + specific properties.
- **SIBLING** ($0.60 < s \leq 0.75$): Classes are related but neither subsumes the other. Query LLM for common parent, create parent node, make both children. If *best_match* already has parent, add C_i^B as sibling under existing parent.
- **NEW_BRANCH** ($s \leq 0.60$): No meaningful similarity. Create C_i^B as independent top-level class (domain-specific concept).

Step 3 - Execute Merging: Initialize $\mathcal{O}_{merged} \leftarrow \text{copy}(\mathcal{O}_A)$, then apply operations for each C_i^B (see Appendix C for complete pseudocode with Neo4j operations).

Transfer Metrics:

Class Transfer Rate (CTR): Fraction of auto classes that reuse existing flood structure:

$$\text{CTR} = \frac{|\text{DIRECT_REUSE}| + |\text{SUBCLASS}|}{|\mathcal{C}_B|} \times 100\% \tag{3}$$

Target: CTR > 60% indicates successful cross-domain transfer.

Property Transfer Rate (PTR): Fraction of auto properties already present in flood:

$$\text{PTR} = \frac{\sum_{C_i^B} |P(C_i^B) \cap P(\text{match}(C_i^B))|}{\sum_{C_i^B} |P(C_i^B)|} \times 100\% \tag{4}$$

Target: PTR > 50% indicates property-level reuse beyond just class structure.

Interpretation: CTR = 60% represents healthy balance—universal insurance concepts (Policy, Claim, Coverage) transfer across domains, while domain-specific concepts (FloodZone, VehicleType) correctly identified as NEW_BRANCH. Extremes are undesirable: CTR = 100% implies no domain differences (unrealistic), CTR = 0% implies zero generalization (system failure).

3.5 Zone 4: Neo4j Storage

Three-layer graph architecture: (1) Ontology classes (20–50 nodes) with SUBCLASS_OF edges, (2) Entity types (50–200 nodes) with INSTANCE_OF_CLASS edges, (3) Entity instances (150K–210K nodes) with domain-specific relations. Indexes on key properties enable $O(1)$ lookup. Natural language interface translates questions to Cypher queries via LLM (see Appendix G for schema details).

4 Evaluation

Automatic: Compare against Riskine reference ontology (`https://github.com/riskine/ontology`) using LLM-based semantic matching (confidence > 0.8). Compute Precision $= \frac{|\mathcal{O}_{induced} \cap \mathcal{O}_{ref}|}{|\mathcal{O}_{induced}|}$, Recall $= \frac{|\mathcal{O}_{induced} \cap \mathcal{O}_{ref}|}{|\mathcal{O}_{ref}|}$ where $\cap$ denotes LLM-judged semantic equivalence. Targets: Precision > 80%, Recall > 70%, F1 > 0.75.

Quality Dimensions: (1) Completeness (Recall vs. Riskine), (2) Correctness (Precision), (3) Consistency (no cycles), (4) Coherence (intra-cluster > 0.6), (5) Utility (query accuracy > 75% on 20 tasks).

Manual: Two domain experts rate 50 classes, 30 hierarchies, 30 extensions as Good/Acceptable/Poor. Cohen's Kappa > 0.7 for agreement. Target: > 80% acceptable. See Appendix E for protocol.

Success: F1 > 0.75 (auto) + > 80% acceptable (manual); CTR > 60%, PTR > 50%; universal concepts > 90% transfer, domain-specific < 10%; outperform LangChain baseline.

5 Implementation

Stack: Neo4j, Llama 3.1 8B (Ollama), DeepSeek-R1 7B, all-MiniLM-L6-v2, graspologic Leiden, LangGraph.

Timeline (11 weeks):

- **Weeks 1–3: Baseline Implementation** — End-to-end LangGraph pipeline, definition of success criteria, and baseline evaluation metrics.
- **Weeks 4–7: Novel Implementation (OpenFEMA)** — Data extraction, fusion, clustering, validation, and schema/ontology extension using the OpenFEMA dataset.
- **Weeks 8–9: Transfer Study** — Cross-domain transfer to auto insurance data, metric computation, and comparison against baseline methods.
- **Weeks 10–11: Polishing & Reporting** — UI refinement, final evaluations, and report preparation.

Deliverables: (1) LangGraph pipeline, (2) Neo4j graphs (flood/auto/merged), (3) Evaluation (CTR/PTR, F1, baseline), (4) Demo (Streamlit), (5) Report (10–15 pages).

References

References

[1] Y. Tiwari et al., "OntoRAG: Enhancing QA through automated ontology derivation," *arXiv:2506.00664*, 2025.

[2] J. Bai et al., "AutoSchemaKG: Autonomous KG construction through dynamic schema induction," *arXiv:2505.23628*, 2025.

[3] B. Zhang and H. Soh, "Extract, define, canonicalize: An LLM framework for KG construction," *EMNLP*, 2024.

[4] D. Edge et al., "From local to global: A Graph RAG approach," *arXiv:2404.16130*, 2024.

[5] Riskine Insurance Ontology, `https://github.com/riskine/ontology`, 2024.

A Related Work Details

OntoRAG: Community detection on entity graphs for ontology induction. Limitation: requires predefined relation types. We extend with cross-LOB transfer measurement.

AutoSchemaKG: Web-scale conceptualization via multi-stage refinement (78K GPU hours). We adopt entity/event distinction but use local LLMs for cost efficiency.

EDC: Open IE with canonicalization. Limitation: flat entity types, no hierarchy. We add type clustering and hierarchical induction.

GraphRAG: Community-based retrieval without formal ontology. We provide explicit schema enabling structured queries and reasoning.

B EVENT Handling and Triple Quality

Events as Relations: Events (actions/occurrences like "filed", "approved") encoded as typed relations between entities. Example: "Claim filed on 2024-01-15" → (Claim C-123, filed_on, 2024-01-15).

Triple Quality Examples:

Quality	Triple	Reason
GOOD	(Policy 5123, coverage_limit, $250K)	Specific, concrete values
BAD	(Policy, has, coverage)	Too generic, no identifiers
MARGINAL	(Policy ABC, has, limit)	Vague relation, needs specificity

Filters: Reject generic relations (is, has, contains). Require specific identifiers and concrete values.

C Schema Extension Algorithm and Examples

C.1 Complete Algorithm Pseudocode

Input: Flood ontology $\mathcal{O}_A$, auto ontology $\mathcal{O}_B$

Output: Merged ontology $\mathcal{O}_{merged}$

```
Initialize: O_merged = deep_copy(O_A)

For each auto class C_i in O_B:
    best_match, score = FindBestMatch(C_i, O_A)

    If score > 0.90:  // DIRECT_REUSE
        Redirect C_i instances -> best_match
        Merge properties: P(best_match) U P(C_i)
        Update type_map[C_i] = best_match

    ElsIf 0.75 < score <= 0.90:  // SUBCLASS
        Verify with LLM: "Is C_i subclass of best_match?"
        If confirmed:
```

```
            Create C_i node in O_merged
            Add edge: C_i -> SUBCLASS_OF -> best_match
            Inherit: P(C_i) = P(best_match) U P(C_i)
        Else: treat as SIBLING

    ElsIf 0.60 < score <= 0.75:  // SIBLING
        If best_match has parent P:
            Add edge: C_i -> SUBCLASS_OF -> P
            (C_i and best_match now siblings under P)
        Else:
            LLM: "Common parent for C_i and best_match?"
            Create parent node
            Add edges: C_i -> parent, best_match -> parent

    Else:  // NEW_BRANCH (score <= 0.60)
        Create C_i as independent top-level class
        No hierarchy connections

Return O_merged
```

C.2 Concrete Execution Examples

Example 1 - DIRECT_REUSE: Auto "PolicyClaim" matches flood "Claim" (score=0.95). *Action:* Merge into existing Claim class, add "vehicleType" property. *Result:* Unified Claim class handles both flood and auto claims.

Example 2 - SUBCLASS: Auto "AutoPolicy" matches flood "FloodPolicy" (score=0.82). LLM: "Both are property insurance policies." *Action:* Create parent "PropertyPolicy", add both as children. *Result:* PropertyPolicy $\leftarrow$ {FloodPolicy, AutoPolicy}.

Example 3 - SIBLING: Auto "Vehicle" matches flood "Property" (score=0.68). LLM: "Common parent is InsurableAsset." *Action:* Create parent, connect both. *Result:* InsurableAsset $\leftarrow$ {Property, Vehicle}.

Example 4 - NEW_BRANCH: Auto "VehicleType" matches nothing (score=0.45). *Action:* Add as independent class. *Result:* VehicleType stands alone (domain-specific to auto).

C.3 Similarity Formula Details

Type Co-occurrence: $\text{cooccur}(t_i, t_j) = \frac{\#(\text{triples with both types})}{\#(\text{triples with either type})}$

Relation Similarity: $\text{rel_sim}(t_i, t_j) = \text{Jaccard}(\text{rels}(t_i), \text{rels}(t_j))$ where rels(t) = set of relation types for instances of type t.

Property Overlap: $\text{prop_overlap} = \frac{|P(C_i) \cap P(C_j)|}{|P(C_i) \cup P(C_j)|}$ (Jaccard of property sets).

D LangChain Baseline Details

Configuration: LLMGraphTransformer with Llama 3.1 8B, direct Neo4j insertion.

Expected Issues: 20–30% entity duplication ("Policy 5123" vs. "policy #5123"), type inconsistency (same concept as "Policy", "InsurancePolicy", "flood_policy"), no ontology queries, no LOB transfer mechanism.

Comparison: Entity resolution F1, type consistency (unique labels per concept), schema coherence, query accuracy on 20 tasks.

E Manual Evaluation Protocol

Annotators: 2 experts with insurance domain knowledge.

Tasks:

1. **Class Quality** (50): Rate name appropriateness, member similarity, property reasonableness (Good/Acceptable/Poor)
2. **Hierarchy Quality** (30): Validate subclass relationships (Correct/Reasonable/Incorrect)
3. **Extension Quality** (30): Verify extension type decisions (Correct/Reasonable/Incorrect)

Agreement: Cohen's Kappa > 0.7 required. Resolve disagreements via discussion.

F LLM Prompt Templates

Open IE:

```
Extract (subject, relation, object) triples. Be specific.
[GOOD] (Policy 5123, effective_date, 2024-01-15)
[BAD] (Policy, has, something)
Text: [chunk]
Output JSON: [{subject, relation, object, confidence}]
```

Subclass Verification:

```
Class A: {name: "FloodPolicy", properties: [...]}
Class B: {name: "AutoPolicy", properties: [...]}
Is B a subclass of A?
Answer: YES/NO/SIBLING
Confidence: 0.0-1.0
Reasoning: [explain]
```

Parent Generation:

```
Given siblings:
- FloodPolicy (covers flood damage to property)
- AutoPolicy (covers auto accidents)
Generate common parent class name and definition.
Output: {name: "...", definition: "..."}
```

G Neo4j Schema Architecture

Three Layers:

```
Layer 3: OntologyClass (20-50 nodes)
  Properties: name, definition, properties[]
  Edges: [:SUBCLASS_OF]
```

```
Layer 2: EntityType (50-200 nodes)
  Properties: name, instance_count
  Edges: [:INSTANCE_OF_CLASS]

Layer 1: Entity Instances (150K-210K nodes)
  Labels: FloodPolicy, Claim, Coverage, etc.
  Properties: id, domain attributes
  Edges: [:FILED_AGAINST], [:HAS_COVERAGE], etc.
```

Indexes: CREATE INDEX ON :FloodPolicy(policy_number), :Claim(claim_id), :OntologyClass(name)

Performance: Lookup $O(1)$ via index, traversal $O(k)$ neighbors, hierarchy $O(d)$ depth ($d \leq 3$).

H Property Type Inference

Algorithm:

```
For each property p across member types:
    values = collect all values for p

    If all(is_numeric(v) for v in values):
        type = numeric
    ElsIf all(matches_date_regex(v) for v in values):
        type = temporal
    ElsIf len(unique(values)) < 20:
        type = categorical
    Else:
        type = string

    If p appears in >80% of types:
        required = True
    Else:
        required = False
```